PRÁCTICA 02 · DESARROLLO COLABORATIVO

TechLoan

Mantenimiento de software heredado con Flutter y GitHub

NexaSoft Solutions · 5 sesiones · Equipos de 3 a 4 integrantes

Objetivo. Asumir el mantenimiento de una aplicación desarrollada por otro equipo, reproducir y corregir errores, incorporar funcionalidades y administrar el trabajo mediante **Issues, ramas, commits, Pull Requests y revisiones de código en GitHub**. Se evaluará tanto el funcionamiento del producto como la colaboración verificable de cada integrante. **No se entregará bitácora diaria.**

1. Contexto de la empresa y alcance

La empresa ficticia *NexaSoft Solutions* recibió el mantenimiento de **TechLoan**, un sistema de préstamo de dispositivos tecnológicos para una institución educativa. La versión existente permite consultar equipos, registrar y devolver préstamos, editar información del catálogo y guardar registros localmente. Sin embargo, el cliente detectó fallas y solicitó mejoras.

El código de partida **no debe reescribirse por completo**. La prioridad es comprender lo existente, identificar el origen de los problemas y realizar cambios pequeños, comprobables y compatibles con otras funciones. Se proporcionan datos ficticios; no se necesita servidor ni base de datos externa.

1.1. Entrega del proyecto base

El profesor proporcionará el archivo `Proyecto_TechLoan.zip` con el código Flutter. Deben extraerlo, instalar dependencias y ejecutar la aplicación. Algunas carpetas nativas de Flutter se generan localmente, como se indica en la sección 3. **Que la aplicación compile no implica que funcione correctamente.**

2. Organización de equipos y responsabilidades

Los equipos estarán integrados por **3 o 4 personas**. Todos deberán trabajar en el **mismo repositorio privado**, pero cada estudiante tendrá una cuenta de GitHub y contribuciones identificables.

- **Coordinación rotativa:** una persona vigila la distribución y el estado de los Issues; el rol puede cambiar diariamente.
- **Responsable de un Issue:** investiga, implementa, prueba y abre el Pull Request asociado. Cada integrante será responsable principal de al menos un ticket.
- **Revisor:** otro integrante revisa el Pull Request, lo prueba o verifica su evidencia antes de integrarlo.

- **Equipo completo:** verifica que la versión final en `main` sea ejecutable y no presente regresiones.

No se evalúa positivamente crear commits sin contenido útil, ni acumular cambios sin integrarlos. El trabajo debe reflejar avances reales durante los cinco días.

3. Preparación y ejecución de Flutter

En la carpeta extraída del proyecto ejecutar:

```
flutter doctor
flutter create . --project-name techloan
flutter pub get
flutter devices
flutter run -d chrome --web-port 7357
```

También pueden usar un dispositivo Android disponible. Para ejecutar las pruebas base:

```
flutter test
```

Importante para pruebas de persistencia web: reutilicen el mismo puerto al cerrar y volver a abrir la aplicación; si cambia la dirección local, el navegador puede utilizar otro almacenamiento y la comparación dejará de ser válida. Para comprobar persistencia, recarguen la misma dirección en el navegador o reinicien usando el mismo puerto.

4. Creación del repositorio dentro de GitHub

La organización oficial es: `Dispositivos-Moviles-UAA-2026`. El profesor ya creó los equipos (por ejemplo, `7B-Equipo-01`). **Nadie debe crear una organización nueva ni trabajar en un repositorio de su cuenta personal.**

4.1. Paso A. Registro de integrantes y acceso

1. Cada persona comunica al profesor su **nombre de usuario de GitHub**, nombre completo, grupo y número de equipo.
2. Acepta la invitación para incorporarse a `Dispositivos-Moviles-UAA-2026`. Comprueba que puede ingresar en https://github.com/Dispositivos-Moviles-UAA-2026.
3. El profesor incorporará a cada alumno al equipo que corresponda. **No pueden agregarse por su cuenta a los equipos**. Si no aparecen los permisos, deben solicitar apoyo antes de trabajar.
4. Elijan **una persona** encargada de crear el repositorio del equipo, una vez que haya aceptado la invitación a la organización.

4.2. Paso B. Crear el repositorio privado

1. En GitHub, presionar + → `New repository`; también se puede entrar desde el botón `New` de la organización.
2. En el campo **Owner** elegir exactamente `Dispositivos-Moviles-UAA-2026`. **No seleccionar la cuenta personal.**

3. Nombre obligatorio: `techloan-7b-equipo-01`, reemplazando `7b` y `01` según su grupo y equipo. Todo en minúsculas.
4. Seleccionar **Private**. No usar `Public`.
5. Dejar sin inicializar el repositorio: **no** crear README, .gitignore ni licencia desde GitHub, porque el ZIP ya los contiene.
6. Presionar `Create repository`. Confirmar que la dirección comience por https://github.com/Dispositivos-Moviles-UAA-2026/.

Permisos y equipos. El profesor asignará al equipo de GitHub permiso `Write` en el repositorio. La persona que creó el repositorio debe compartir la URL para que el profesor verifique la pertenencia a la organización, la visibilidad `Private` y el acceso de los demás integrantes. No es necesario otorgar permisos administrativos a los alumnos.

4.3. Paso C. Subir el proyecto original

La persona responsable del repositorio entra a la carpeta `Proyecto_TechLoan`, una vez ejecutada la preparación de Flutter, y usa estos comandos (ajustar la URL al equipo):

```
git init -b main
git add .
git commit -m "chore: importar proyecto base TechLoan"
git remote add origin https://github.com/Dispositivos-Moviles-UAA-2026/techloan-7b-equipo-01.git
git push -u origin main
```

Si la carpeta ya es un repositorio Git, no repitan `git init`; revisen primero `git status` y `git remote -v`. **Este commit inicial de importación no cuenta como desarrollo individual.** Los demás integrantes clonan el repositorio mediante:

```
git clone https://github.com/Dispositivos-Moviles-UAA-2026/techloan-7b-equipo-01.git
```

Después ejecutan `flutter pub get` y comprueban que puedan iniciar la aplicación.

5. Tickets oficiales del cliente

Los seis requerimientos son obligatorios. Deben registrarse como **seis Issues independientes en GitHub**, asignar responsables y mantener su estado actualizado. Los tickets `BUG` describen resultados que no se comportan correctamente; los `FEAT` describen funcionalidad nueva.

BUG-101 · Control de disponibilidad Corrección · Prioridad alta

Reporte: el cliente afirma que algunos equipos pueden prestarse aunque no tienen unidades disponibles.
Resultado esperado: cuando las unidades disponibles son cero, no debe registrarse un nuevo préstamo ni generarse una cantidad negativa. Un préstamo válido debe disminuir exactamente una unidad.
Aceptación: demostrar, con un caso reproducible, que la operación inválida no crea registros ni modifica el inventario; comprobar también un préstamo correcto.

BUG-102 · Inventario al devolver — Corrección · Prioridad alta

Reporte: al devolver ciertos equipos, el registro del préstamo cambia de estado pero la disponibilidad no aumenta como corresponde.
Resultado esperado: toda devolución válida regresa exactamente una unidad del tipo de equipo prestado; no deben permitirse efectos dobles al intentar devolverlo otra vez.
Aceptación: probar como mínimo dos categorías distintas y explicar cómo verificaron que el total es consistente.

BUG-103 · Edición de catálogo — Corrección · Prioridad media

Reporte: al modificar información de un equipo después de ordenar o buscar el catálogo, ocasionalmente cambia otro registro.
Resultado esperado: guardar debe modificar solamente el dispositivo que se eligió; su identificador debe conservarse y los demás registros no deben alterarse.
Aceptación: editar un equipo visible después de utilizar la búsqueda, comprobar su ID y revisar que otros equipos mantuvieron sus datos.

BUG-104 · Persistencia de préstamos — Corrección · Prioridad alta

Reporte: algunos préstamos recién registrados desaparecen después de cerrar y volver a abrir la aplicación.
Resultado esperado: todo préstamo guardado debe permanecer, tanto si contiene observaciones como si éstas se dejan vacías.
Aceptación: registrar casos con y sin observaciones, reiniciar sobre el mismo almacenamiento local y verificar ambos; no deben desaparecer registros anteriores.

FEAT-201 · Filtros de préstamos — Mejora · Prioridad media

Solicitud: el cliente necesita filtrar el listado de préstamos por **estado** (Todos, Activo, Devuelto) y por **categoría de dispositivo** (Todas y las categorías existentes).
Resultado esperado: ambos filtros deben funcionar **simultáneamente**; permitir restablecer la consulta; reflejar cambios tras registrar o devolver préstamos.
Aceptación: probar al menos tres combinaciones de filtros y la opción de ver todos.

FEAT-202 · Préstamos vencidos — Mejora · Prioridad media

Solicitud: mostrar en el inicio el **número de préstamos activos vencidos** y una sección sencilla con equipo, responsable y fecha límite de los vencidos.
Regla de negocio: un préstamo está vencido si sigue **activo** y su fecha límite es anterior a la fecha actual. Un préstamo devuelto nunca cuenta como vencido. El vencimiento no cambia automáticamente el estado del préstamo.
Aceptación: validar con un préstamo pasado activo, uno pasado ya devuelto y uno activo todavía en plazo.

6. Forma de trabajar en GitHub

6.1. Crear y administrar los Issues

Desde el repositorio: `Issues` → `New issue`. El título deberá comenzar con el identificador del ticket, por ejemplo `[BUG-101] Impedir préstamos sin disponibilidad`. El contenido debe incluir:

- **Qué sucede** (o qué funcionalidad se necesita).
- **Cómo reproducirlo** (para errores) o casos de uso (para funcionalidades).
- **Resultado esperado** y una lista de verificación de aceptación.
- **Persona asignada**, etiqueta `bug` o `enhancement`, y referencias al Pull Request cuando exista.

Los Issues se actualizarán con hallazgos relevantes y se cerrarán cuando la corrección haya sido integrada y probada. Si descubren otros errores, pueden registrarlos como Issues adicionales, pero **no sustituyen los seis tickets obligatorios**.

6.2. Crear una rama por cambio

No desarrollar directamente sobre `main`. Una rama por ticket permite integrar y revisar los cambios de manera independiente. Ejemplo:

```
git switch main
git pull origin main
git switch -c fix/bug-101-disponibilidad
# Realizar cambios y comprobar la aplicacion
git add .
git commit -m "fix: validar inventario antes del prestamo"
git push -u origin fix/bug-101-disponibilidad
```

Nombres sugeridos: `fix/bug-102-devoluciones`, `fix/bug-103-edicion`, `feat/feat-201-filtros`. Los mensajes de commit deben describir **qué cambió**; eviten `arreglo`, `cambios`, `avance` como mensajes únicos.

6.3. Pull Requests y revisión entre compañeros

1. Al terminar un cambio, abrir `Pull requests` → `New pull request` y comparar la rama del ticket contra `main`.
2. Incluir en la descripción: **qué cambió**, **cómo se probó** y la referencia al Issue real de GitHub. Por ejemplo `Closes #3` si el Issue correspondiente tiene el número `3`. *El número de Issue no es el mismo que BUG-103.*
3. Solicitar revisión a **un compañero distinto del autor**. Revisar el cambio, ejecutar o comprobar las pruebas y dejar una observación concreta en GitHub.
4. Una vez validado, integrar el PR a `main`. Después actualizar la copia local: `git switch main` y `git pull origin main`.
5. Si existen conflictos de código, deben resolverse colaborativamente antes de integrar; describan brevemente en el PR qué se decidió.

Regla de trabajo de la clase: todo cambio funcional se integra mediante Pull Request revisado por otro integrante. En repositorios privados del plan GitHub Free puede no estar disponible forzar esta regla mediante protección de ramas; **es obligatoria**

como procedimiento académico, incluso si la plataforma permite saltarla.

7. Plan de trabajo de cinco días

Día	Actividades centrales	Evidencia en GitHub
1	Aceptar invitaciones, confirmar equipos, crear repositorio privado, importar el proyecto, ejecutarlo, reproducir fallas y repartir responsabilidades.	Repositorio bajo la organización; integrantes habilitados; commit base; seis Issues abiertos y asignados; ramas de trabajo iniciales.
2	Investigar y corregir errores de inventario, edición o persistencia.	Commits descriptivos, comentarios técnicos en Issues y primeros Pull Requests cuando corresponda.
3	Continuar correcciones e implementar mejoras solicitadas; coordinar cambios compartidos.	Ramas `feat/...`, pruebas, PR con descripción y revisión por un compañero.
4	Integrar modificaciones, resolver conflictos, comprobar escenarios y corregir regresiones.	PR revisados, Issues cerrados al integrar, nuevas correcciones documentadas si hacen falta.
5	Prueba general, revisión de criterios de aceptación y entrega del producto.	`main` ejecutable, historial de PR/Issues, README actualizado, entregas individuales identificables.

8. Uso responsable de inteligencia artificial y confidencialidad

Pueden usar IA para **explicar conceptos, proponer estrategias de prueba, interpretar mensajes de error genéricos y generar ejemplos ficticios**. Como simulación de una empresa, **queda prohibido pegar el proyecto completo, archivos internos no autorizados, secretos, datos de clientes o credenciales en herramientas externas sin aprobación**. Incluso si los datos entregados son ficticios, se evaluará que actúen con criterio profesional.

Para solicitar apoyo a una herramienta pública, describan el comportamiento general y creen un ejemplo mínimo con nombres y datos inventados, sin revelar código confidencial. **Toda respuesta de IA debe verificarse con pruebas locales.** Cuando una decisión técnica provenga de una sugerencia de IA, basta con declararlo brevemente en la descripción del PR: propósito de la consulta, tipo de información compartida y verificación aplicada. No es necesaria una bitácora adicional.

9. Pruebas, criterios de entrega y calificación

Cada ticket debe incluir evidencia **proporcionada al Issue o al PR**: pasos de reproducción, pruebas realizadas y, cuando ayude, una captura de pantalla. No se exige video, Word ni PDF diario. El proyecto debe arrancar sin errores de compilación y mantener la funcionalidad previa.

Criterio	Puntos
Correcta solución de los 4 bugs y las 2 funcionalidades	40
Uso verificable de Issues, ramas, commits, PR y revisiones	30
Pruebas, ausencia de regresiones, claridad técnica y README	20
Participación individual sustentada en la actividad del repositorio	10
Total	**100**

La funcionalidad del proyecto es una responsabilidad grupal; la participación se revisará por alumno. **Un estudiante sin aportaciones propias, participación en Issues o revisiones no recibirá automáticamente la calificación de su equipo.** La cantidad de commits por sí sola no equivale a calidad.

9.1. Checklist final

☐ Repositorio privado bajo `Dispositivos-Moviles-UAA-2026`, con el nombre correcto.

☐ Los 3 o 4 integrantes tienen acceso y su participación es identificable.

☐ Los seis Issues obligatorios tienen responsable y estado actualizado.

☐ Se usaron ramas específicas, PR y revisiones entre compañeros.

☐ `main` puede ejecutarse, con evidencia de pruebas y sin regresiones conocidas.

☐ El `README.md` contiene integrantes, ejecución, funciones terminadas y problemas pendientes.

☐ El profesor dispone de la **URL del repositorio**; no se entrega ZIP final como sustituto del historial.

Entrega oficial: un representante envía al profesor la URL del repositorio dentro de https://github.com/Dispositivos-Moviles-UAA-2026/, cuyo nombre debe ser `techloan-<grupo>-equipo-<numero>`. **No habrá bitácora diaria:** la evidencia de avance será GitHub (Issues, commits, ramas, PR, revisiones y pruebas). Cada integrante deberá haber trabajado desde su propia cuenta.